Sky Cob: Building and Flying UWOSLAB's Recoverable High-Altitude Balloon Payload

Project record for the `cobject` repository · INAV 9.0.1 / EdgeTX 2.11.5 / CadQuery · October 14, 2026 · Status: WIP

TL;DR

- **What it is:** Sky Cob is a recoverable weather-balloon payload built around a **SpeedyBee F405 Wing** flight controller running **stock INAV 9.0.1**. It carries GPS, an ExpressLRS receiver, an analog camera and a high-power VTX, and it lives inside a 3D-printed **corn-cob shell** with parachute loops on the tip. Nothing in the payload is flown: the FC is used for telemetry, OSD, blackbox logging and GPS. It is not used for control.
- **The most important configuration fact:** the FC is set up so that it **never tries to come home or land**: `failsafe_procedure = NONE`, `nav_rth_allow_landing = NEVER`, `nav_rth_climb_first = OFF`, `safehome_usage_mode = OFF`. A balloon drops out of RC range as a matter of course, and it should stay armed and keep logging when that happens.
- **The enclosure is now generated, not hand-modeled:** `skycob_cob.py` reads the stack STLs (chassis, FC plate, RX/VTX plate) and builds the cob cavity around them. It writes three STEP files: the cob, a separate access door, and the chassis with M3 standoffs that plug into the cob's ceiling. If a stack part changes, re-run the script and the shell follows.
- **There is one inconsistency to fix before the next flight:** the README's channel table, the FC `aux` lines and the radio mixer disagree about OSD and beeper channels. The FC expects the beeper on **CH9**, and the radio sends nothing there. See Caveats.

Key Findings

The system in one paragraph

Ground side: an EdgeTX radio (Radiomaster Boxer is preferred) with an external ELRS module (Radiomaster Nomad is preferred) runs the `Sky Cob.yml` model. It shows CRSF telemetry (RSSI, GPS, altitude, battery, attitude, flight mode), and a laptop ground station plugs into the module's USB for tracking. Air side: the F405 Wing FC is mounted pitched 90° (`align_board_pitch = 900`). It takes GPS from a uBlox M10-class unit (HGLRC M100 Pro or Foxeer M10), RC and telemetry over a DBR4 ELRS receiver, and overlays OSD onto an analog camera that points straight down. It transmits video on Raceband channel 8 (5917 MHz). Power is a 3S/12 V pack, and a separate 5G GPS tracker gives a recovery fix that does not depend on the FC. README.md INAV CLI dump

The FC configuration is a delta on stock INAV, not a firmware fork

`INAV_9.0.1_cli_SKY_COB_20260701_003845.txt` is a `diff all` batch: `batch start` → `defaults noreboot` → settings → `save`. Flash the stock `SPEEDYBEEF405WING` target, then load the dump in the CLI tab. The companion **sky-cob-configurator** (a balloon-only fork of INAV Configurator) can push the same baseline with one "Apply Balloon Defaults & Reboot" button. It was verified against the real board: 98 settings applied with zero errors, and a `diff all` after the reboot confirmed every one. docs/weather-balloon-configurator.md

Calibration is the only part that does not carry over between boards. `acczero_*`, `accgain_*` and `gyro_zero_*` were measured on the original unit, so recalibrate any new FC in standard board orientation, not in its mounted orientation. The configurator's balloon defaults leave calibration out on purpose.

The enclosure geometry comes from the parts inside it

The cob cavity is the bounding envelope of the three stack STLs, grown by a clearance on each face. Every number is a constant at the top of `skycob_cob.py`, so changing a clearance means editing one line and re-running the script. The chassis top bears directly on the cavity ceiling (0 mm ceiling clearance). Four 7 mm standoffs on the chassis plug up into sockets in the cob top, and M3 screws go down into them from outside through 7 mm access bores in the tip. skycob_cob.py

Printability shaped the shell's appearance

The "kernels" are rounded square tiles on a 32-facet barrel, not a smooth cylinder. Each tile's underside is chamfered at 45°, so the cob prints upright with no overhangs. The faceted barrel also keeps the STEP file manageable: plane/sphere intersections are exact circles, while cylinder/sphere intersections become B-splines that made the file several times larger. Generating the model still takes about 9 minutes, and the cob STEP is about 46 MB.

Details

Repository contents

File	Role
`INAV_9.0.1_cli_SKY_COB_…txt`	FC configuration batch for INAV 9.0.1 (`SPEEDYBEEF405WING`)
`Sky Cob.yml`	EdgeTX 2.11.5 model: CRSF module, 16 channels, telemetry screens

`cobby19mm_cam_stand.stl`	Main chassis; 19 mm camera bracket at the bottom
`cobbyspeedy-top.stl`	FC cover/mount plate for the F405 Wing
`cobby20x40.stl`	RX/VTX X-plate (20×20 holes onto 40×40 rear standoffs)
`cobshell.stl`, `fix model.stl`	Previous hand-modeled shell (upright, and reoriented for a 250×250 bed)
`skycob_cob.py`	CadQuery generator for the three STEP files below
`skycob_cob.step`	Corn-cob shell: cavity, sockets, access bores, loops, side holes, RP-SMA port, door opening
`skycob_door.step`	Removable back-wall door (0.3 mm undersize)
`cobby19mm_cam_stand_standoffs.step`	Chassis as a solid with 4 M3 standoffs and M2 pilot holes in the back pegs

Shell geometry (from the generator constants)

Feature	Value
Stack envelope (chassis + plates)	≈ 51 × 44 × 166 mm
Cavity clearance	10 mm sides and FC face; 25 mm back/peg face; 20 mm at the mouth; 0 at the ceiling
Cavity	≈ 71 × 79 × 186 mm, rounded corners, opens at the bottom
Core wall / barrel	4 mm minimum wall; core radius ≈ 53 mm; ≈ 113 mm across the kernels
Kernels	32 columns, 10.5 mm pitch, 3.5 mm proud, 1 mm grooves, 1.5 mm fillets
Tip	Elliptical dome, 0.8 × core radius long
Overall height	≈ 230 mm body; ≈ 250 mm to the top of the parachute loops (estimate)
Parachute loops	3 tori at x = −30 / 0 / +30 mm, 14 mm radius, 10 mm cord section
Standoffs / sockets	Ø7 mm × 20 mm, 0.4 mm socket clearance, M3 pilot 2.5 mm, 5 mm ceiling plate
Side vents	4 × Ø5 mm radial holes at 45° offsets, mid-height of the RX/VTX plate
RP-SMA bulkhead	40 mm below the side holes, through the door; Ø6.5 mm hole, 2.5 mm panel, Ø12 mm spot-faces
Access door	Full cavity width, from the base to 5 mm above the RX/VTX plate; 4 × 4 mm side rails; flush push-in fit

Flight controller settings that matter

Area	Setting
Safety (balloon-specific)	`failsafe_procedure = NONE`, `failsafe_min_distance_procedure = NONE`, `nav_rth_allow_landing = NEVER`, `nav_rth_climb_first = OFF`, `safehome_usage_mode = OFF`
GPS	`gps_sbas_mode = WAAS`; Galileo, BeiDou and GLONASS on. Expect a lock in 30 s to 3 min from cold; launch with at least 5–8 satellites
Orientation	`align_board_pitch = 900`, `align_mag = CW270FLIP`, `small_angle = 180` (arms at any attitude)
Video	`vtx_band = 5` (Raceband), `vtx_channel = 8` = 5917 MHz, full power
Blackbox	Rate denominator 2; logs `NAV_POS NAV_PID MAG ACC ATTI RC_DATA RC_COMMAND MOTORS SERVOS`; microSD, only 4 GB usable
Identity / locale	`name = SKY COB`, `pilot_name = UWOSLAB`, `osd_units = IMPERIAL`, `tz_offset = -240`, `tz_automatic_dst = USA`

Channel mapping as it actually is

Channel	Radio source (`mixData`)	FC mode (`aux`)	Range (µs)
CH5 (AUX1)	SD	ARM	1300–2100
CH6 (AUX2)	SA	OSD OFF	1300–2100
CH7 (AUX3)	SB	OSD ALT 1	1300–1700
CH8 (AUX4)	SC	OSD ALT 2	1700–2100
CH9 (AUX5)	— (no mix)	BEEPER	1300–2100

The README instead lists OSD Off and both OSD pages on CH6/SC and the beeper on CH7/SB. Its setup text says "Arm … Aux 5" where the FC actually uses AUX1, which is CH5.

Assembly order

1. FC into its bay, correctly oriented; FC cover on top with M2.5 screws, with no wiggle.
2. GPS on the top platform with VHB tape or glue. Heading doesn't matter because its compass isn't used.
3. RX and VTX bolted through the 20×20 holes of the X-plate with M2. The subassembly goes on the rear 40×40 standoffs with the RX facing in and the VTX facing out.

4. 5G tracker on the rear of the chassis between the two large holes, and the 3S battery on the other side.
5. Analog camera on the 19 mm bracket, pointing straight down.
6. Slide the chassis up into the cob until its standoffs seat in the ceiling sockets, then drive 4 M3 screws down through the tip's access bores. Fit the door last.

Recommendations

1. **Reconcile the channel map first.** Pick one source of truth, then update the README table, the FC `aux` lines and `Sky Cob.yml` together. The quickest functional fix is to add a CH9 mix for the beeper switch, or move `aux 4` to an existing channel. A beeper you can't trigger makes ground recovery harder.
2. **Test-print the risky interfaces before the full cob.** These are the standoff-to-socket fit (0.4 mm clearance), the zero-clearance ceiling bearing, and the door on its rails (0.3 mm undersize). Cut test sections from the STEP so you don't commit to a multi-day print.
3. **Check the cob's height in the slicer.** By estimate it is close to 250 mm with the loops. If it exceeds the printer's Z, reduce `DOME_RATIO` or `LOOP_RAISE` and regenerate.
4. **Add door retention.** The door currently just pushes in. A single screw boss, a snap tab or tape is enough, but it has to hold through a parachute opening.
5. **Retire the old shell STLs once the STEP cob is validated.** Then update the README's print list (it still says to print `fix model.stl`) and its repository table.
6. **Publish a configurator release.** A pre-built Windows package of sky-cob-configurator lets non-developers set up an FC without Node.js or the CLI dump.

Caveats

- **Everything is WIP:** none of the generated parts has been printed and test-fit at the time of writing.
- **Dimensions are derived:** the envelope and height figures above are computed from the STLs and generator constants, not measured from a print.
- **The chassis STEP is faceted:** it was converted from a mesh, so it has no true cylinders or planes to snap to in CAD, and it is about 28 MB.
- **File sizes:** `skycob_cob.step` (≈ 46 MB) is close to GitHub's 50 MB warning threshold. More kernel detail or features could push it over.
- **Calibration is per board:** never reuse the dump's accelerometer and gyro offsets on a different FC.

- **RF:** the VTX power and band choice must comply with local regulations and the VTX's unlock procedure. Check your VTX manual.
- **Known configurator bug:** `js/appUpdater.js` throws on startup. The cause is upstream, and the bug is harmless for setup.